CyCrew: AI-Assisted SIEM Incident Response

Case study of an internship project | Deepak Singh, AI Engineer Intern | Oct 2025 - Jan 2026

> **Scope and confidentiality.** This document describes the design, my part in it and its current limits. Every alert, IP address, username and log line shown here is synthetic sample data (documentation-range IPs such as 203.0.113.45). It contains no company logs, customer information or company source code.

1. Overview

The problem

A Security Operations Center (SOC) receives alerts from its SIEM (the system that collects and correlates security logs). Each alert has to be read, matched to a kind of attack and turned into response steps, mostly by hand. Asking a general-purpose LLM to do this directly is risky because it can produce confident remediation steps that are not part of any vetted procedure.

Intended users

SOC analysts doing first-pass triage, who need a consistent, structured starting point (what happened, how confident the analysis is, what to do next) that they then review. The tool supports the analyst; it does not act on systems by itself.

Objective

Build a prototype that accepts either of two inputs and returns a structured incident report with response steps:

- **Classified alert** (a SIEM alert with an `alert_id` and an alert name): matched to a known attack playbook, and the LLM writes its response plan from that playbook.
- **Unclassified logs** (a raw block of log text): analysed by the LLM alone, which decides whether an attack occurred and what type it is, citing evidence from the logs.

Technology at a glance

Component	Technology	Role
Dashboard	Vue 3, Vue Router, Bootstrap 5, Vite	Submit alerts or logs, view the incident feed
Orchestration	n8n (11 nodes, 2 webhooks)	Routes each input, calls RAG and LLM, stores results
Retrieval (RAG)	FastAPI, FAISS, BAAI/bge-small-en	Returns the best-matching attack playbook
LLM	Gemini 2.5 Flash-Lite via n8n	Writes the incident analysis as structured JSON
Result store	FastAPI, in-memory list	Holds incidents for the dashboard

2. System architecture

The dashboard never talks to the AI services directly. It posts JSON to one of two n8n webhooks; n8n orchestrates retrieval and the LLM call, writes the result to a small store, and the dashboard polls that store every 5 seconds.

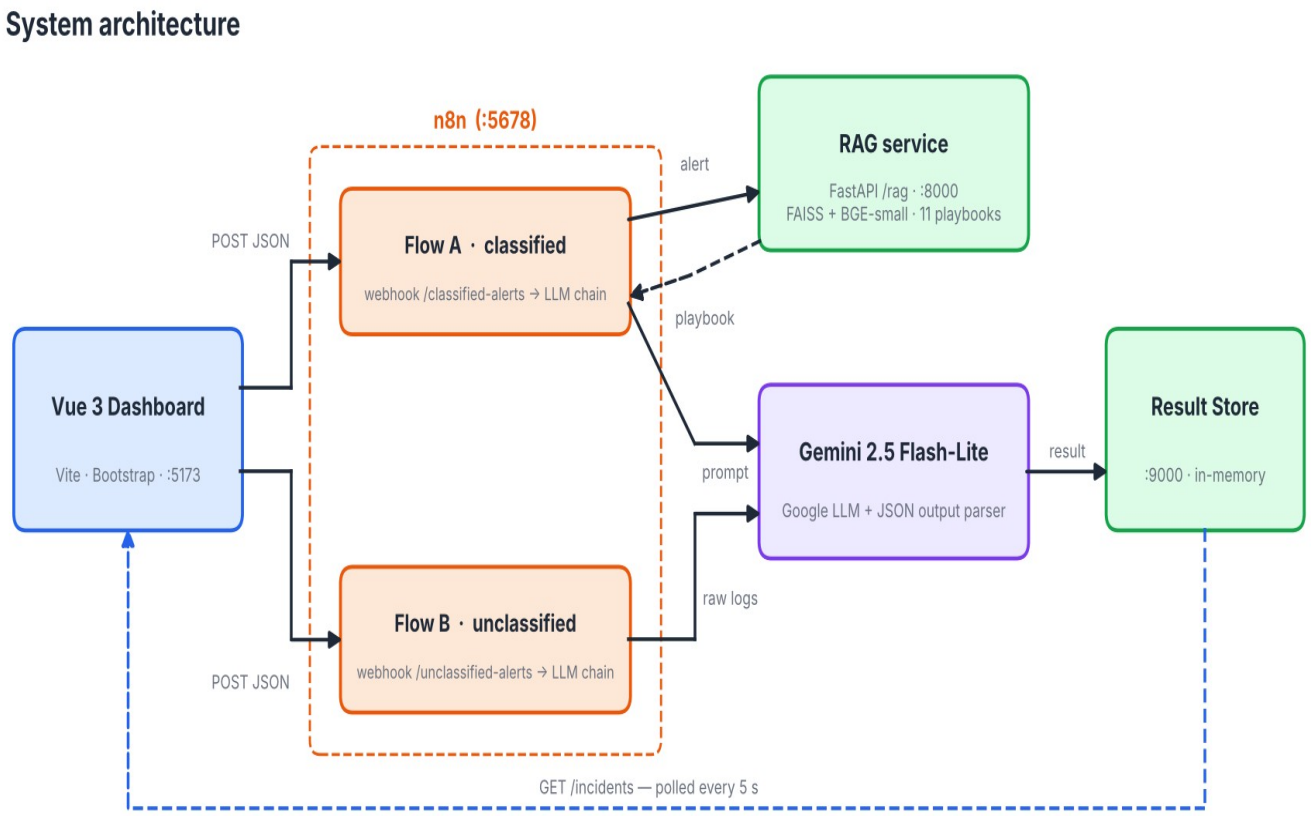


Figure 1. End-to-end architecture. Flow A (classified) is grounded by the RAG service; Flow B (unclassified) goes straight to the LLM.

Layer	What it does	Where
Interface	Parses pasted JSON, picks the webhook by shape (`alert_id` or `logs`), polls and renders incident cards	Vue app: Analyze.vue, Incidents.vue, IncidentCard.vue
Orchestration	Two independent flows, each webhook to LLM chain to store; webhooks reply with the store's response	n8n workflow
Retrieval	Embeds the alert name, returns the nearest of 11 playbooks (mitigation and outage-recovery steps)	rag/: retriever.py, embeddings.py, build_index.py
LLM	Gemini 2.5 Flash-Lite with a Structured Output Parser: attack_detected, attack_type, confidence, mitigation_steps	n8n Basic LLM Chain nodes
Backend	POST /rag for retrieval; POST /store-incident and GET /incidents for storage	FastAPI: main.py, result_store.py

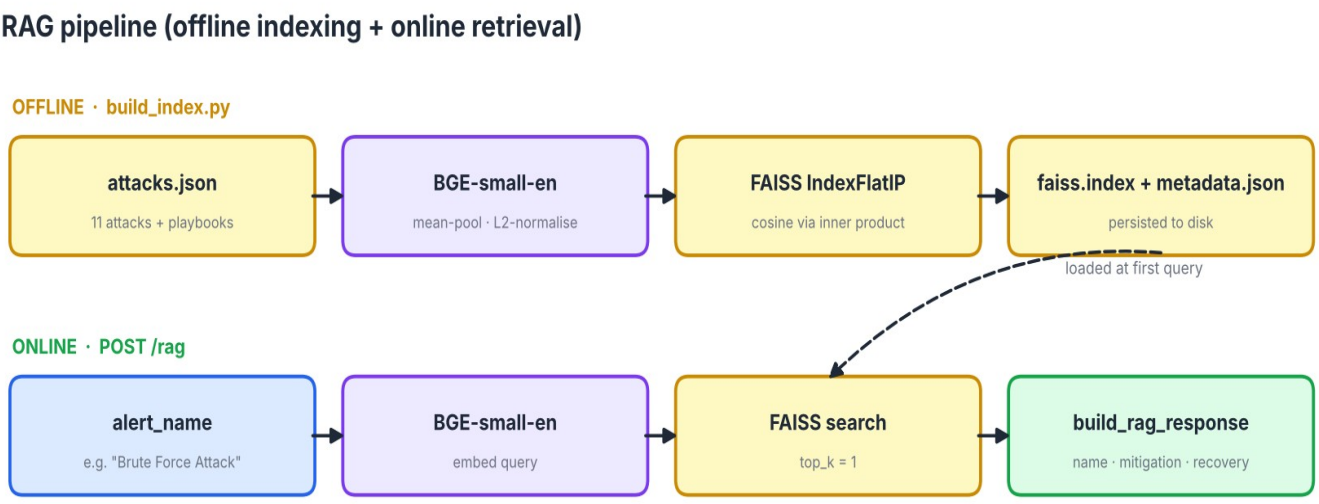


Figure 2. Retrieval pipeline. Offline: playbook names are embedded and indexed. Online: the alert name is embedded and the top-1 neighbour is returned.

Retrieval details

Each attack name in `attacks.json` is embedded with BAAI/bge-small-en (mean-pooled, L2-normalised, 384 dimensions) and stored in a FAISS `IndexFlatIP`. On unit vectors, inner product equals cosine similarity, so the search is a semantic match rather than a keyword rule: 'Brute Force Attack' or a close variant reaches the right playbook. The knowledge base covers 11 attacks: phishing, malware, ransomware, DoS, DDoS, SQL injection, XSS, man-in-the-middle, brute force, privilege escalation and data exfiltration.

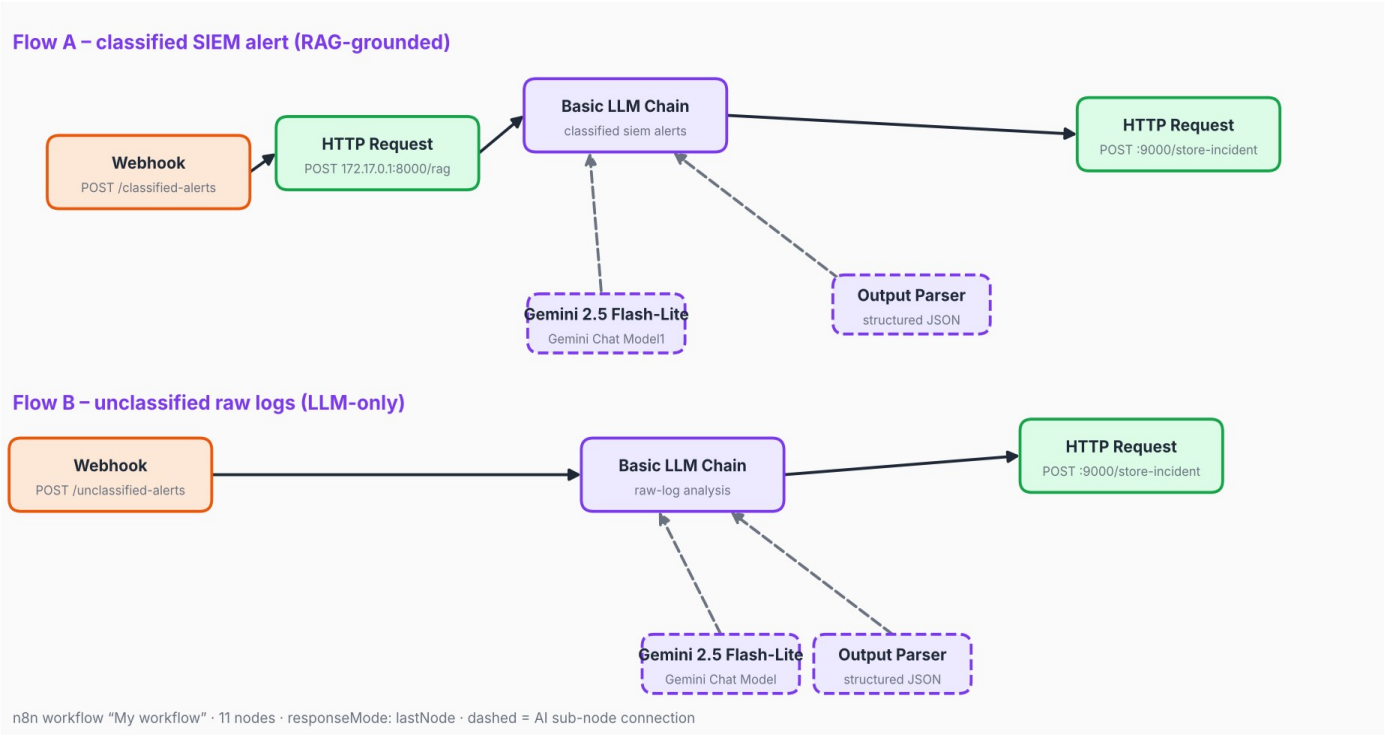


Figure 3. The n8n workflow (exported JSON, drawn as two flows). Dashed lines are AI sub-node connections.

3. My contributions

I worked on the project end to end. The table lists what I designed and built, and the decision behind each part.

Module	What I did	Decision and reason
RAG service (retriever, embeddings, build_index, generator)	Built the offline index and the online POST /rag endpoint; structured the 11-playbook knowledge base with mitigation and outage-recovery steps per attack.	Semantic search over names instead of keyword rules, so variants of an alert name still resolve. Index loads lazily on first query.
n8n workflows	Built both flows: webhook, RAG call (Flow A), Gemini chain, store call. Wrote the prompts and the structured JSON output schema.	Two flows rather than one, because known alerts can be grounded in a playbook and raw logs cannot.
Prompt design	Flow A: the retrieved playbook is the source of truth; no steps beyond it. Flow B: act as a SOC analyst, use only the logs, cite IPs, users and timestamps, do not assume an attack on weak evidence, do not invent CVEs or tools.	Constrain the model by what it may use, rather than hoping it stays accurate.
Result store	FastAPI app with POST /store-incident and GET /incidents; stores source, original input, result and timestamp.	In-memory for speed of prototyping; limits listed in section 6.
Vue dashboard	Two views (Analyze, Incident Feed), one card component, router; input routing by payload shape; 5-second polling.	A single text area accepts both input types, so analysts do not choose a mode.

Note: this section is written from the project's code and workflow. Replace or trim any row that was built by a teammate so the page claims only what I personally did.

4. Technical challenges

Challenge 1: keeping LLM output tied to vetted procedure

Problem. An LLM asked for incident response steps will happily write plausible ones, including steps that are not in any approved playbook. In a security setting that is worse than no answer.

Approach. For classified alerts, retrieval runs first and the playbook is injected into the prompt. The system message tells the model to treat that context as the source of truth, to organise the answer as Immediate Actions, Mitigation and Recovery, to reference the specific IP, user and timestamp from the alert, and not to add steps beyond the context. A Structured Output Parser forces a fixed JSON shape so the dashboard can render it without parsing free text.

What this did not solve. Retrieval always returns the nearest playbook, even for an alert that matches nothing (for example 'Port Scan'), and the model is told to trust it. This is the most important open issue (section 6).

Challenge 2: two kinds of input, one interface

Problem. Some inputs are structured alerts with a known name; others are unstructured log text with no label. They need different handling but analysts should not have to choose.

Approach. The dashboard inspects the pasted JSON: `alert_id` goes to the classified webhook, `logs` goes to the unclassified webhook, anything else is rejected. Both flows end in the same output schema and the same store, tagged `siem_classified` or `siem_unclassified`, so one feed shows both. For raw logs the prompt explicitly allows 'no attack detected' and forbids inventing evidence.

5. Evaluation and results

There are no measured accuracy or time-saving results yet. The prototype was demonstrated on a small number of sample inputs and has not been benchmarked. The table separates what was exercised from what was only reviewed.

Part	How it was checked	Result
Result store + dashboard	Run locally; sample incidents posted to the real store; screenshots taken from the running app	Works: both views render, the feed lists incidents, polling updates it
RAG response shape	Derived by reading attacks.json and the code for an exact-name query ('Brute Force Attack')	Expected output documented; not captured from a live run
n8n workflow + Gemini	Reviewed from the exported workflow JSON; the model credential is referenced by name only	Structure confirmed; the report environment had no n8n or Gemini key, so no live end-to-end run was recorded here
Retrieval quality, report correctness, triage time	Not evaluated	No metrics

Test inputs

Classified alert:

```
{ "alert_id": "ALR-2026-0142", "alert_name": "Brute Force Attack", "severity": "High",
"timestamp": "2026-01-12T10:42:11Z", "source_ip": "203.0.113.45", "target_user": "admin" }
```

Unclassified logs:

```
{ "logs": "Jan 12 10:41:02 srv01 sshd[2211]: Failed password for root from 198.51.100.23 port 51122
ssh2\nJan 12 10:41:09 srv01 sshd[2213]: Accepted password for admin from 198.51.100.23 port 51133
ssh2" }
```

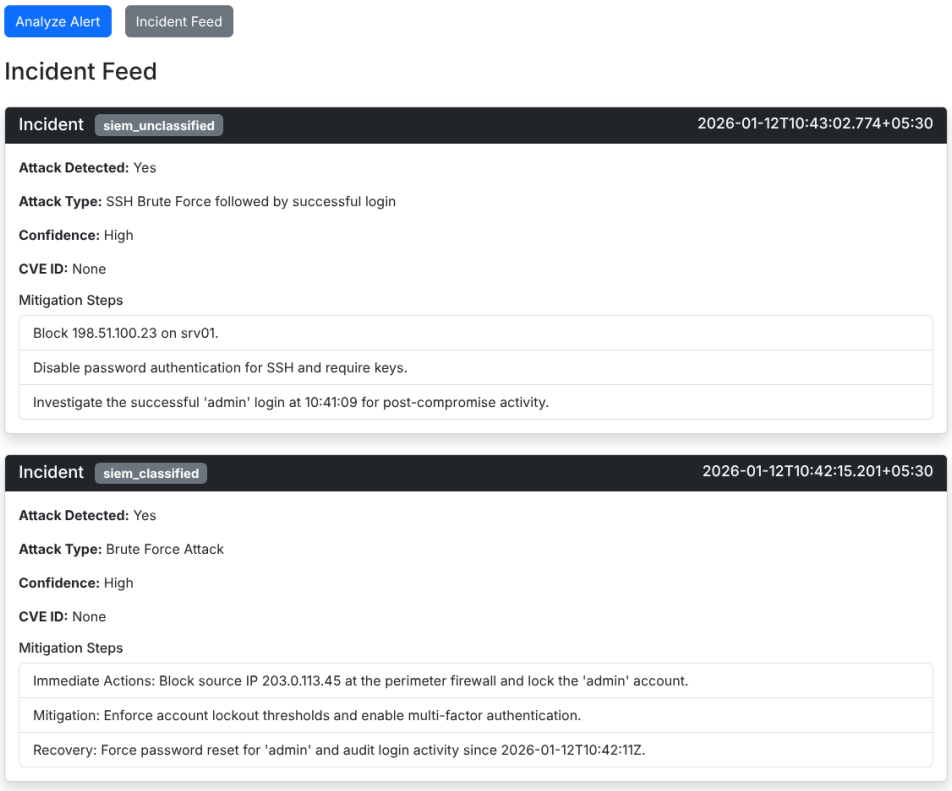


Figure 4. Incident Feed with two sample incidents (one per path). The incident data is sample data posted to the real result store.

Evaluation plan (not yet done)

- **Retrieval:** top-1 accuracy on a labelled set of alert names, including vendor-style variants (for example 'Multiple failed logins') and out-of-scope names, reporting how often an out-of-scope alert is wrongly matched.
- **Report correctness:** a rubric scored by a reviewer: is the attack type right, are IPs, users and timestamps quoted correctly, is any step outside the playbook.
- **Usefulness:** time for an analyst to reach a first action with and without the tool, on the same set of alerts.

6. Limitations and learnings

The items below come from reviewing the code and workflow. They are ordered by importance.

#	Limitation	Impact	What I would do
1	Retrieval returns the nearest playbook with no similarity check, so the 'No playbook found' branch in main.py can never run.	An unknown alert (e.g. 'Port Scan') is given a wrong playbook that the LLM trusts.	Return the FAISS score and reject matches below a threshold; route rejects to the log-analysis flow.
2	Retrieval uses only alert_name against 11 attack names.	Vendor-specific names may map badly.	Embed descriptions or aliases per attack as well as names.
3	The incident card shows a CVE ID, but the output schema has no cve_id field.	The card always shows 'None'.	Add the field to the schema or remove it from the card.
4	In the classified chain, system-message placeholders lack the n8n expression prefix.	They reach the model as literal text; only the main prompt carries real values.	Remove them or convert to {{ }} expressions.
5	The result store is an unbounded in-memory list with no authentication.	Incidents vanish on restart; memory grows; open if exposed.	SQLite or Postgres, a size cap, an API key.
6	No success or error feedback on submit; input is cleared even on failure; the feed has no error handling.	An analyst cannot tell whether an alert was processed.	Status banners and try/catch around polling.
7	URLs and ports are hard-coded (127.0.0.1:9000, :5678, 172.17.0.1).	Works on one machine layout only.	Vite .env and n8n environment variables.
8	requirements.txt is a full pip freeze including CUDA packages.	A multi-GB install for a CPU-sized model.	Keep direct dependencies and a CPU torch build.
9	Unused Vue template files remain; main.py uses the deprecated alert.dict().	Clutter and a Pydantic warning.	Delete leftovers; use model_dump().

What remains uncertain

- How accurate the Gemini analysis is across real, noisy alerts: it has not been measured.
- Whether name-only retrieval holds up beyond the 11 stored attacks and against vendor naming.
- Whether analysts find the output actionable; no analyst feedback was collected.

What I learned

- Grounding is only as good as retrieval: a confident wrong match is worse than no match, so a rejection path matters as much as the match itself.
- Constraining a model by what it may use (the playbook, or the logs only) is easier to reason about than asking it to be careful.
- Fixing the output schema early made the interface simple and exposed mismatches (such as the

missing CVE field) quickly.

- Without even a small labelled test set, quality claims cannot be made. I would build one first next time.

Appendix: running the prototype

Start in this order: (1) RAG service on port 8000: `uvicorn main:app --host 0.0.0.0 --port 8000`; (2) result store on 9000: `uvicorn result_store:app --host 0.0.0.0 --port 9000`; (3) n8n on 5678: import the workflow, attach a Google Gemini credential to both model nodes, activate; the workflow calls 172.17.0.1 (Docker host), so the APIs must listen on 0.0.0.0; (4) dashboard on 5173: `npm install`, `npm run dev` (Node 20.19+ or 22.12+).